

Research-Based Lesson Plan:

Business Programming with JavaScript in Web Applications

John McSwain, jmcswain3@sc.edu

The University of South Carolina

EDET 709: Applications of Learning Principles, Section J71

Dr. Lucas Vasconcelos

December 8, 2022

Instructional Problem Statement

Undergraduate business students enrolled in Georgia Institute of Technology's Scheller College of Business (SCB) may choose to complete an Information Technology Management (ITM) concentration while completing their degree coursework. The ITM concentration allows students to enroll in elective courses which bridge conceptual and practical gaps between business problems and technology competence. *MGT 3745 Business Programming* is an ITM course that provides opportunities to learn programming languages to design, develop, and deploy digital solutions to solve business problems in practice (e.g., product development management). Undergraduate business students are not required to be skilled in coding and programming, which could lead to occupational competitive disadvantages with businesses that have begun or completed digital transformations into technology-oriented business models. MGT 3745 provides undergraduate business students with marketable exposure to various information system topics (e.g., database design, distributed system design, analytics, automation, artificial intelligence, and web/mobile/cloud application development). MGT 3745's goal is to increase students' professional value, bargaining power, and negotiating leverage when seeking employment opportunities. Because many programming languages fall into unary contexts, MGT 3745's lessons introduce the programming language JavaScript, due to JavaScript's widespread use in several business domains and industries.

Target Audience

The target audiences for the MGT 3745 are undergraduate junior (<30 cr. hr. remaining) and senior (<15 cr. hr. remaining) level students enrolled in MGT 3745 during the Fall or Spring term of each academic year. The typical class size is between 17-25 students per semester and assumes students have no prior coding skill proficiency, but are competent with basic computing skills (opening/reading/saving files) and Internet aptitude. Occasionally, a small number of students, typically less than 15%, register for the course with some pre-existing knowledge or experience in JavaScript, if not another programming language.

Instructional Model

The instructional model chosen for the business programming lesson is Gagne's Nine Events of Instruction.

Explanation of Instructional Model

Robert Gagné believed that learning requires a series of transformations through nine events of instruction. Each stage, internal processes, learning actions, and example actions are listed below (Driscoll, 2005):

Instructional Model Details: Gagné's Nine Events of Instruction			
Instructional Event	Internal Process	Action	Techniques
Gaining attention	Reception of patterns	Stimulus change	Initial instructions with visual cues
Informing learners of the objective	Expectancy, executive control	State the intended outcome	Short, simple statements
Stimulating recall of prior learning	Retrieval of prior working memory	Reuse of prior knowledge and/or skills	Recalling previous concepts
Presenting content	Selective perception	Distinctive content	Definitions
Providing guidance	Semantic encoding via retrieval cues	Organization advisement or management	Examples within context
Eliciting performance	Response organization	Issue performance orientated tasks	Provide tasks
Providing feedback	Establishing reinforcement	Provide informative and actionable feedback	Information about (in)correct responses

Assessing performance	Retrieval activation & enabling reinforcement	Additional performance tasks with feedback	Include new concepts and examples
Enhancing retention and transfer	Providing retrieval cues; generalization	Issue varied practice and review	New examples with varied forms and circumstances

(Gagné et al., 1992)

Instructional Problem and Model Relationship

Effective transfer with cognitive methodologies, namely Gagné's nine events of instruction, is critical in allowing business students to harness self-confidence in learning JavaScript. While a small percentage of students have an innate ability to understand the code due to a 'geek gene', others struggle to overcome losses in learning momentum when topics are introduced later in the course (Lister, 2011). Student success in business programming correlates to students who develop 'coding grit', the "...ability to persevere and focus through computer programming activities" (Mahatanankoon & Wolf, 2021). Furthermore, student success in business programming correlates with student computer self-efficacy (CSE), the belief in one's ability to control a specific outcome with a computer, which is determined dynamically in a course by four factors: performance accomplishment, vicarious experience, verbal persuasion, and physiological states (Mahatanankoon & Wolf, 2021). There are intersections of Gagné's nine events of instruction, which includes multiple varied performance activities, numerous assessments, contextual feedback, and recalling working memory with increasing student 'coding grit' and CSE, and thereby increasing probable student success.

In addition, business programming with JavaScript utilizes computational thinking as a framework for solving problems. Several components of computational thinking (e.g., abstraction, decomposition, pattern recognition, build algorithms, test, assessment) intersect

several Bloom’s Taxonomy of Cognitive Outcome counterparts (e.g., comprehension, application, analysis, synthesis, evaluation) (Markandan et al, 2022).

Lesson Plan Objectives

Objective	Bloom’s Taxonomy of Cognitive Outcomes
Students will be able to create or modify the functional behavior and data of amCharts time-based web applications	Comprehension Understanding the nature and function of the amCharts time-based web components Application Using JavaScript syntax to develop amCharts components and generate data compatible for display in business practice, particularly product performance
Students will be able to apply formatting, structural, filtering, and mapping rules with JavaScript Object Notation (JSON)	Knowledge Revisiting JSON will allow students to add contextual value to the contents and mutations of text-based key/value pairs

Lesson Plan Details: Resources

Students are allowed to utilize any combination of approved listed course resources to study and gain generic understanding of JavaScript programming syntax, coding practices (e.g., keywords, built-in methods, data types, application programming interfaces), and coding tools.

- W3Schools
 - <https://www.w3schools.com/js/default.asp>
- Mozilla Developer Network
 - <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
 - https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web
- Eloquent JavaScript
 - <https://eloquentjavascript.net/>
- Replit (collaborative integrated development environment)
 - <https://replit.com/>

Lesson Plan Details: Nine Events of Instruction Model + ARCS Model of Motivation			
Event	Instructor	Students	ARCS
Gain Attention	Instructor opens an interactive website showing an interactive web application visualizing time-based data using JavaScript within amCharts : https://www.amcharts.com/demos-v4/timeline-v4/	Students observe the clickable and draggable components of the demo, and generate questions about how/where the application can be customized for their purposes	Concreteness (Attention) Instructor will show examples of job listings requesting proficiency in data visualization
Communicate Objectives	<u>Instructor notes:</u> <i>In this lesson, we'll discuss the importance of visualizing data to convey compelling insights about how a web-based product/project is performing (usage), using the same language used to develop the web application: JavaScript.</i>	Students listening to instructor	-
Stimulating Recall of Prior Learning	<u>Instructor notes:</u> <i>Reflecting on JSON (JavaScript object notation), how can we organize data in such a way that allows an open-source library like amCharts to interpret, iterate, and display our product's usage data? What kind of metadata can be inferred from the display (e.g., color, labels, filtering)</i>	Students share their experience with JSON on prior exercises to display data using loops and conditional code blocks	-
Presenting Stimulus	Instructor explains the mechanisms used within the demo, including JSON data model, metadata, and amCharts event-based code example(s)	Students listen and ask questions about different parts of the code and data, mostly for clarification, and to validate questions on code variations.	Present Worth (Relevance) Instructor will show industries and salaries seeking data visualization skill
Providing Learning Guidance	<u>Instructor notes:</u> <i>We'll make some changes to the data to see how modifications to the chart labeling, color theme, data categories, chart type, and even values themselves affect the display of the application. Next, we'll make some event-based changes to the code to react to various types of user interaction (e.g., clicks, mouse-over)</i>	Students observe and attempt to make similar, if not identical, changes within their own web browsers to validate their comprehension.	-

Eliciting Performance	<u>Instructor notes:</u> <i>Now, let's break into working groups and see if everyone understands the underlying concepts of amCharts timeline types. Break into groups of 3-4 students, and attempt to take an existing chart example, and make it look and behave with custom behaviors and data (shows the expected output onscreen)</i>	Students break into working groups and discuss and modify the code collectively toward matching the modified example. Some students will understand but may/will need to explain their reasoning to others in their group.	<i>Self-confidence (Confidence)</i> Students can feel confident when communicating their thoughts and understanding to their peers and are comfortable to discuss without perceived failure
Providing Feedback	Instructor will check in on coding and discussion progress for the duration of the in-class exercise and provide guidance where applicable, refraining from providing "the answer" vs guiding toward "an answer"	Students continue to work together, openly discussing various approaches to matching the shown example.	-
Assessing Performance	After the exercise time has elapsed, a member of each group will share their coding with the class to illustrate their progress, including their computational thinking and practical approaches.	Students vocalize their code, and instructor asks follow-up questions on the pros and cons of each step.	<i>Positive Outcomes (Satisfaction)</i> Instructor verbally praises well defined approaches along with guiding multiple student driven solutions
Enhancing Retention and transfer	Instructor provides an individual assignment to be completed over the course of one week (6-7 calendar days), which expands the learning objective of learning amCharts by requiring each student to develop a timeline from scratch (no scaffolded code provided) to ensure comprehension of each aspect of the chart and the function of each code block. Students will be expected to provide commented explanations of their code and conform to ideal coding practices, such as Single-responsibility Principle , Modular Programming , and Separation of Concerns	Students work for the next week individually to complete the assignment by reading amCharts documentation and revisiting coding examples to develop personalized coding solutions and deliver an acceptable application with concrete behavioral and design requirements	-

Assessment Components

The following rubric will be used to assess each student's individual code submission functionality, organization, testability, and documentation.

Criteria	1	2	3
Functionality (consistency, errors)	Non-functional; significant logical or runtime errors;	~50% functional; few code blocks work properly; inconsistent behavior; intermittent errors	~100% functional; produces expected output; consistent behavior
Code Style (organization, naming, redundancy)	Low/no styling used; lacks organization; poor naming standards; low/no code reusability	~50% organized; moderate naming standards; partial code reusability	~100% styling used; organized in reusable modules; naming standard match
Testability	Lacks any controllable input(s) to verify expected or anticipated output(s)	~50% testable; partially permits controlled input(s) & verifiable output(s)	~100% testable; permits controlled input(s) & verifiable output(s)
Documentation (comments, explanations, references)	Low/no documentation; poor semantic, explanatory, contextual, or referential info	~50% readable; partial explanations, contexts, & references provided	~100% readable; complete explanations, contexts, & references provided

Instructional Model Defense

Gagné's nine events of instruction are the most appropriate instructional model because the MGT 3745 Business Programming course requires students to learn intellectual and cognitive skills, and the nine events of instruction provide ideal learning conditions (Driscoll, 2005). For example, Gagné and Driscoll defined optimal conditions for learning intellectual skills, which include directed attention, cues for proper order and execution of procedural steps, and varied examples of organizing the learner's retention and transfer (Driscoll, 2005). Gagné and Driscoll also defined optimal conditions for learning cognitive strategies, which include summarizing topics, reflection opportunities, and the provision of actionable feedback to inform learners if their strategic efforts are practical (Driscoll, 2005).

Compatible instructional models with the MGT 3745 Business Programming course include project- and problem-based learning. Problem-based learning requires a learner-centered approach, collaboration, analysis, and self/peer assessments are excellent mechanisms for students to develop their business programming problem-solving skills while empowering their peers to learn and self-correct when necessary. (Savery, 2006). In addition, project-based requires focus on the development of an artifact, which in the case of business programming would likely be a digital artifact, would provide learners with the opportunity to work in groups collaboratively, explore and investigate approaches, and establish milestones and checkpoints for timely assessment and evaluation (Grant, 2002).

Weaknesses with the nine events of instruction include the potential overuse of instructional events, which can lead to boredom. Underuse of events of instruction can lead to inadequate or misdirected learning (Driscoll, 2005). In addition, different learning objectives may require variations of the events of instructions that are different throughout the course and may lead to lowered student satisfaction or frustration.

References

- Driscoll, M.P. (2005). *Psychology of Learning for Instruction*. Pearson.
- Gagné, R.M., Briggs, L.J., & Wager, W.W. (1992). *Principles of instructional design*. Fort Worth, TX: Harcourt Brace Jovanovich College Publishers.
- Grant, M. M. (2002). Getting a grip on project-based learning: Theory, cases and recommendations. *Meridian: A middle school computer technologies journal*, 5(1), 83.
- Harasim, L. M. (2012). *Learning theory and online technology*. Routledge.
- Keller, J. M. (1987). Development and use of the ARCS model of instructional design. *Journal of instructional development*, 10(3), 2-10.
- Lister, R. (2011). Computing education research geek genes and bimodal grades. *ACM Inroads*, 1(3), 16-17.
- Mahatanankoon, P., & Wolf, J. (2021). Cognitive Learning Strategies in an Introductory Computer Programming Course. *Information Systems Education Journal*, 19(3), 11-20.
- Markandan, N., Osman, K., & Halim, L. (2022). Integrating Computational Thinking and Empowering Metacognitive Awareness in Stem Education. *Frontiers in psychology*, 13, 872593. <https://doi.org/10.3389/fpsyg.2022.872593>
- Savery, J. R. (2006). Overview of Problem-based Learning: Definitions and Distinctions. *Interdisciplinary Journal of Problem-Based Learning*, 1(1). <https://doi.org/10.7771/1541-5015.1002>